



THE SCHOOL OF AI



NAVTTC

Working offline

Day 1 Extended — uv, VS Code and Jupyter on your own machine. The same code as Colab, with no internet, no uploads, and nothing of your client's leaving the laptop.

BEFORE WE START

You are not leaving Colab

You are adding a second way of working. From today you choose which one a job needs.



Keep using Colab for

Class labs and homework.
Anything you want to share or show.
Quick tests from a phone or a borrowed laptop.
Work that does not touch private data.



Use your own machine for

Client files that must not be uploaded.
Work during load-shedding or with no internet.
Scripts that run every morning.
Anything you are being paid for.

One skill, two places to use it. The code you write does not change.

Three hours, one setup, forever

Slow the first time and two minutes every time after. These are the four commands you will use for years.

1 Hour 1 · The tool

Why offline, which editor, and installing uv — the one tool that replaces four older ones.

2 Hour 2 · The project

Installing Python, creating a project, and giving it its own private environment.

3 Hour 3 · Running things

A script in the terminal, a notebook in VS Code, and the same file opened back in Colab.

Everything on this sheet is typed by hand. Still no pasting.

Colab runs on Google's computer. At some point you need your own.

The moment a client says the file must not leave your laptop, or the internet goes at nine in the evening, the browser tab stops being an option.

What each one is good at

	COLAB	YOUR OWN MACHINE
Internet needed	Yes, always	Only to install things the first time
Setup time	None	Forty-five minutes, once
A client's private file	Uploaded to Google	Never leaves your laptop
Runs while you sleep	No, the session closes	Yes
Installing packages	Every session, again	Once, and it stays
Best for	Learning, sharing, demos	Real client work

You need both. This is not a competition.

Three names, one editor underneath

Cursor and Antigravity are built on VS Code. Every instruction today works in all three — only the AI panel differs.



VS Code

The one this course uses. Free, everywhere, and every tutorial you find assumes it.



Cursor

VS Code with a stronger AI assistant built in. Paid beyond a free tier.



Antigravity

A newer agent-first editor, also built on VS Code. Same menus, same shortcuts.

The Colab extension for VS Code still needs the internet. That is not offline.

Install uv

One command. Check the official page first — installers change: docs.astral.sh/uv

COPY THE COMMAND FROM THE OFFICIAL PAGE, NOT FROM A PHOTOGRAPH OF THIS SLIDE

Windows, in PowerShell

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

macOS or Linux, in Terminal

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Then CLOSE the terminal, open a new one, and check:

```
uv --version
```

Closing the terminal is not optional. It is why uv is not found.

One tool instead of four

Older tutorials will tell you to install Python, then pip, then virtualenv, then remember which one you used. uv does all of it.



Installs Python

Any version, without touching the one your operating system depends on.



Makes projects

One command creates the folder, the settings file and the version marker.



Manages packages

uv add writes the package into your project and records the exact version.



Runs your code

uv run uses the project's own Python, whether or not you activated anything.

Install Python, and pick a version

3.13 IS THE SAFE CHOICE TODAY. 3.14 IS FINE TOO

```
uv python install 3.13
```

```
uv python list          # what uv can see on this machine
```

```
uv run python --version # what this project would actually use
```

```
uv python pin 3.13      # fix this project to one version
```

Do not chase the newest release.

Libraries take months to support a brand-new Python, and you cannot yet tell a library problem from your own mistake.

Create your first project

ONE FOLDER PER PROJECT, ALWAYS. NEVER TWO PROJECTS IN ONE FOLDER

```
cd Desktop          # or Documents, or E:\work – anywhere you will find it
```

```
uv init openchat --python 3.13
```

```
cd openchat
```

```
# Now look in the folder. uv made several files for you.
```

Choose where it lives, deliberately.

A project on the Desktop gets deleted by accident. Make one folder for all your course work and keep them together.

What each of those files is

FILE	WHAT IT IS	DO YOU EDIT IT?
pyproject.toml	The project's identity card: name, Python version, packages	Yes, or let uv add do it
.python-version	Which Python this project uses	Rarely, by hand
main.py	A starter file with a hello-world in it	Yes — or make your own
README.md	Where you describe what the project does	Yes, and you should
uv.lock	Exact versions, so it installs the same elsewhere	Never. uv owns this
.venv	This project's own private Python and packages	Never open it

Its own Python, kept separate

ACTIVATING IS OPTIONAL — UV RUN WORKS WITHOUT IT

```
uv venv                                # make the environment

.venv\Scripts\Activate.ps1            # Windows PowerShell
.venv\Scripts\activate.bat            # Windows CMD
source .venv/bin/activate              # macOS or Linux

uv add ipykernel                       # the piece Jupyter needs
```

PowerShell refusing? Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

Open the folder, add two extensions

The editor opens a folder, not a file. Everything you do from here happens inside that project.

- | | | |
|---|--------------------------------|---|
| 1 | Open it | Type code with a dot in the terminal. If that is not recognised, open VS Code and use File, then Open Folder. |
| 2 | The Python extension | By Microsoft. It gives you the run button, the error underlines and the environment picker. |
| 3 | The Jupyter extension | By Microsoft. It lets notebooks run inside the editor, exactly like Colab. |
| 4 | Optional: an icon theme | Makes file types easy to spot at a glance. Cosmetic, and worth the thirty seconds. |

Your first script on your own machine

SAME CODE AS LAB 01. SAME OUTPUT. NO INTERNET

```
# New file: main.py – type it, do not paste
name = "Bilal"
city = "Faisalabad"
print("Hello,", name + "!", "Welcome to", city + "!")
```

```
# Terminal inside VS Code: Ctrl + `
uv run app.py
```

Turn off your wifi and run it again.

It still works. That is the whole point of this session, and it is worth doing once so you believe it.

Notebooks inside VS Code

Same cells, same Shift and Enter, same file format. The only new step is choosing which Python runs it.

1 Make the notebook

New file named day01.ipynb. VS Code opens it with cells instead of plain text.

2 Select the kernel

Top right, Select Kernel, then Python Environments, then the one showing .venv inside your project.

3 Markdown then code

A markdown cell for your heading, a code cell for the work. Shift and Enter runs the cell you are in.

4 Check what you picked

If the kernel is not the .venv one, you are running a different Python and your packages will be missing.

A notebook goes everywhere

This is why the format is worth learning. One file, and four different ways to hand it over.



Saved with output

The results are stored inside the file. Whoever opens it sees what you saw.



Back into Colab

Upload the same ipynb and it runs unchanged. Nothing to convert.



Exported to PDF

For a client or an assignment. HTML always works if PDF asks for extra pieces.



Into a repository

It is one text file underneath, so it belongs in version control like any code.

Seven things that go wrong

WHAT YOU SEE	WHAT TO DO
uv: command not found	Close every terminal, open a new one
PowerShell: running scripts is disabled	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
code . not recognised	Open VS Code, then File and Open Folder
No kernel to select	uv add ipykernel, then reload VS Code
Notebook runs the wrong Python	Click the kernel name, choose the one with .venv in the path
ModuleNotFoundError for a package you added	Wrong folder or wrong kernel. Check both, in that order
Downloads keep failing	Run the same command again — uv resumes rather than restarting

Set it up, prove it works

Follow the extended worksheet. Write down every command that failed and what fixed it.

1	Install and check	uv installed, terminal reopened, uv --version printing something. Python 3.13 installed through uv.
2	Make the project	openchat created, opened in VS Code, ipykernel added, both extensions installed.
3	Run the script	app.py printing the same output as your Colab notebook. Then turn the wifi off and run it again.
4	Run the notebook	day01.ipynb with the .venv kernel, the name-cleaner working, saved with its output.
5	Export it	PDF or HTML, and upload the same ipynb back into Colab to prove it runs in both.

END OF DAY 1 EXTENDED

Four commands, and the machine in front of you is a workshop.

uv init, cd, uv add, uv run. Every project in this track from here on starts exactly the same way — and none of it needs anyone's permission or connection.